

Lab of Web Technologies

Comprehensive Study Guide

Academic Year 2025-2026

Ca' Foscari University of Venice

Professor: Zeynep Yücel

Complete course coverage: Internet Fundamentals, HTML, CSS, JavaScript, jQuery, Bootstrap, AJAX, HTTP, Flask, Jinja2, Cookies & Sessions, Security, Databases & SQL

Includes: Full Mock Exam with Answers (10 Multiple Choice + 6 Open Questions)

Prepared for: Alessio Manera — Group 11
September 2026 Exam Session

Table of Contents

1. Exam Structure & Rules	
2. The Internet & World Wide Web	<i>Lectures 1</i>
3. HTML — HyperText Markup Language	<i>Lectures 2-4</i>
4. CSS — Cascading Style Sheets	<i>Lectures 5-7</i>
5. JavaScript	<i>Lectures 8-10</i>
6. jQuery, Bootstrap & AJAX	<i>Lecture 11</i>
7. HTTP Protocol & Flask Web Server	<i>Lecture 12</i>
8. Cookies, Sessions & Authentication	<i>Lecture 13</i>
9. Web Security	<i>Lecture 13</i>
10. Databases & SQL	<i>Lectures 14-15</i>
11. Jinja2 Templating	<i>Lectures 11-12</i>
MOCK EXAM — Questions & Answers	

1. Exam Structure & Rules

Key Concept: The exam has two components: **50% Written Test** (5 multiple-choice + 3 open questions) and **50% Project + Oral Presentation**. You need at least **18/30** in each to pass.

Key Deadlines (from exam date)

- **4 weeks before:** Submit group members
- **3 weeks before:** Submit 1-page project proposal on Moodle
- **2 weeks before:** Receive feedback; revise if needed
- **1 week before:** Submit final project archive (.zip/.tar.gz) with documentation
- **Exam day:** Oral presentation of the web application

Prerequisites

- Must pass 'Introduction to Coding and Data Management' before taking the exam
- Both written and oral exams must be passed within the same academic year

■ **Exam Tip:** The written exam is based **strictly on the course slides**. This study guide covers all slide content. Focus on understanding concepts, not memorizing code verbatim.

2. The Internet & World Wide Web

2.1 What is the Internet?

The Internet is the **global system of interconnected computer networks** that use the **Internet Protocol Suite (TCP/IP)** to link devices worldwide. It consists of private, public, academic, business, and government networks, all interconnected together.

- A **computer network** allows nodes (computers) to share resources (exchange data)
- Each node is identified by an **address** and generally referred to as a **host**
- Two hosts may exchange data even without a direct connection (different network topologies)

Communication Protocols

A **communication protocol** is a set of rules for exchanging information over a network. Protocols are usually **layered in a stack** — each protocol leverages the services of the layer below it.

The TCP/IP 5-Layer Model

Layer	Name	Responsibility
5	Application	Services and apps (HTTP, email, FTP). Handles authentication & encryption.
4	Transport	Reliable end-to-end communication, flow control, error correction (TCP/UDP).
3	Network	Routing packets to destination, addressing, congestion avoidance (IP).
2	Data Link	Framing data, error detection/correction between two nodes (Ethernet).
1	Physical	Physical transmission of bits via cables, connectors, hardware.

2.2 Internet ≠ World Wide Web

The **Internet** is the infrastructure (routers, cables, servers) using TCP/IP. The **World Wide Web (WWW)** is an information system of interlinked hypertext documents accessed via the internet using web browsers. The WWW primarily uses **HTTP/HTTPS** protocols.

Architectural Bases of the WWW

- **URLs** identify resources
- **Standardized protocols** (HTTP) enable interaction via request-response messages
- **Resources** have representations (MIME types) interpreted by browsers

URL Structure

```
scheme://user:password@host:port/path;params?query#fragment
```

- **Scheme:** Protocol to use (http, https, ftp)
- **Host:** DNS domain name or IP address (must always be specified)
- **Port:** TCP port (default 80 for HTTP, 443 for HTTPS)
- **Path:** Location of resource on the server
- **Query:** Key-value pairs (e.g., ?item=1234#=3)
- **Fragment:** Identifies a section within the resource

URI vs URL vs URN

- **URI** (Uniform Resource Identifier): General term encompassing both URLs and URNs
- **URL** (Uniform Resource Locator): Identifies AND locates a resource
- **URN** (Uniform Resource Name): Names a resource without specifying location

2.3 Evolution of the Web

- **1989**: Tim Berners-Lee proposes the WWW at CERN
- **1990**: Three foundational technologies created: HTML, URI, HTTP
- **1993**: NCSA Mosaic browser — first to show text and images together
- **1995**: JavaScript created (included in Netscape Navigator 2)
- **1996**: CSS born — separate content from presentation
- **2005**: AJAX coined — asynchronous web applications

Web 1.0 vs Web 2.0 vs Web 3.0

Web 1.0	Web 2.0	Web 3.0
Static pages	Dynamic, user-generated content	Machine-readable data (Semantic Web)
Read-only	Read-write (participatory)	Peer-distributed, AI-driven
Publishers → Users	Users create content (social media)	Machines interpret data

Key Concept: Web 3.0 (Semantic Web) aims to make Internet data **machine-readable**. It goes beyond client-server toward a peer-distributed system. (Exam Q1 topic!)

3. HTML — HyperText Markup Language

3.1 Fundamental Concepts

HTML is a **markup language** (not a programming language) used to define the **content and structure** of web documents. It is **declarative** — it defines document structure but pages are not interactive without JavaScript.

Language Types Comparison

Type	Characteristics	Examples
Programming	Compiled, creates software apps	C, C++, Java, C#
Scripting	Interpreted, automates tasks, adds functionality	JavaScript, Python, PHP, Perl
Markup	Interpreted by browser, annotates documents	HTML, XML, Markdown

Key Concept: HTML defines **structure**. CSS handles **presentation**. JavaScript handles **behavior**. This separation is a fundamental design principle.

3.2 Basic HTML Template

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="utf-8" />
<title>Page Title</title> </head> <body> Hello world
</body> </html>
```

- **<!DOCTYPE html>** — Declares HTML5 (not a tag!)
- **<html>** — Root element, surrounds entire document
- **<head>** — Contains meta info (title, charset, CSS/JS links)
- **<body>** — Contains the actual visible content
- Elements consist of **opening and closing tags**, can be nested in a tree structure
- Every element (except html) has exactly **one parent**

3.3 Block vs Inline Elements

Block Elements	Inline Elements
Change content flow (line breaks)	Do NOT change content flow
Occupy full available width	Take only needed width
Can contain other blocks/inline	Found inside block elements
<div>, <h1>-<h6>, <p>, , , , <nav>, <form>, <table>	, <a>, , , , <i>, , <small>, <button>

3.4 Important HTML Elements

Text Structure

- **<p>** — Paragraph (block element with automatic spacing)
- **<h1> to <h6>** — Headings (most to least important)
- **
** — Manual line break (inline, no closing tag)

- **<pre>** — Preserves whitespace formatting

Lists

- **** — Ordered list (numbered)
- **** — Unordered list (bullet points)
- **<dl>**, **<dt>**, **<dd>** — Definition list with terms and descriptions
- List items **** are block elements and can contain nested lists

Tables

```
<table> <tr> <th>Header 1</th> <th>Header 2</th>
</tr> <tr> <td>Cell 1</td> <td>Cell 2</td>
</tr> </table>
```

Links and Images

- **text** — Anchor tag for hyperlinks
- URLs can be **absolute** (full path) or **relative** (from current document)
- **** — Embed images (requires src and alt)

Semantic vs Non-Semantic Tags

- **Semantic:** **<header>**, **<nav>**, **<article>**, **<section>**, **<footer>**, **<figure>**, **<time>**, ****, ****
- **Non-semantic:** **<div>**, ****, ****, **<i>**, **<u>** — tell nothing about content meaning

Common Attributes

- **id** — Unique identifier (for CSS/JS targeting)
- **class** — Semantic class membership (can be shared by multiple elements)
- **style** — Inline CSS rules
- **data-*** — Custom attributes in HTML5 (must start with 'data-')

The <head> Section

- **<title>** — Required, sets browser tab text
- **<meta>** — Character set, viewport, etc.
- **<link>** — Links external resources (stylesheets)
- **<base>** — Sets base URL for relative links
- **<script>** — Links or embeds JavaScript

HTML Forms

```
<form action="/submit" method="POST"> <label for="name">Name:</label>
<input type="text" id="name" name="name" required> <input type="submit"
value="Submit"> </form>
```

- **action** — URL where form data is sent
- **method** — HTTP method (GET or POST)
- **Input types:** text, password, email, checkbox, radio, file, submit, button, range, datetime
- The **for** attribute of **<label>** should match the **id** of its **<input>**
- The **name** attribute determines the key sent to the server

4. CSS — Cascading Style Sheets

4.1 Fundamentals

CSS describes how HTML elements are displayed. It **separates content from presentation**, saves work by controlling layout of multiple pages at once, and accounts for different devices.

Key Concept: CSS is the answer to Exam Q2: its purpose is **to separate content from presentation**.

Three Ways to Apply CSS

- **External stylesheet:** <link rel="stylesheet" href="style.css"> (recommended)
- **Embedded:** <style> block in <head>
- **Inline:** style attribute on elements (not recommended)

4.2 CSS Syntax

```
selector { property: value; property: value; }
```

- **Selector:** Identifies which elements the rules apply to
- **Declaration block:** Contains property-value pairs inside { }
- **Comments:** /* comment */

4.3 Selectors

Selector Type	Syntax	Example
Element	element	h1 { color: red; }
Class	.classname	.warning { font-weight: bold; }
ID	#idname	#para1 { color: red; }
Universal	*	* { margin: 0; }
Descendant	A B	div p { color: red; }
Child	A > B	body > p { color: black; }
Pseudo-class	A:state	a:hover { color: yellow; }
Attribute	A[attr]	p[lang="fr"] { ... }
Grouped	A, B	h1, h2 { color: gray; }

4.4 Specificity (Critical for Exam!)

When multiple rules conflict, **specificity** determines which wins:

- **Inline styles:** 1,0,0,0 (1000 points)
- **ID selectors:** 0,1,0,0 (100 points)
- **Class/pseudo-class/attribute:** 0,0,1,0 (10 points)
- **Element selectors:** 0,0,0,1 (1 point)
- **Universal selector (*):** 0 points

Specificity Examples

Selector	Specificity
h1	0,0,0,1
p em	0,0,0,2
.grape	0,0,1,0
p.bright em.dark	0,0,2,2
#id216	0,1,0,0
inline style=""	1,0,0,0

Cascade Rules

- 1. Find all rules matching an element
- 2. Sort by **specificity** (higher wins)
- 3. If equal specificity, **later declaration wins**
- **!important** overrides all other specificity rules
- **Inherited values** have NO specificity (not even zero)

4.5 The Box Model

Every element generates a rectangular box with these layers (inside-out):

- **Content:** The actual content (width × height)
- **Padding:** Space between content and border
- **Border:** Lines surrounding padding (border-width, border-style)
- **Margin:** Space outside the border

Box-model properties are NOT inherited.

4.6 Positioning

Position	Description
static	Default. Element flows normally in the document.
relative	Offset from normal position; original space preserved.
absolute	Removed from flow; positioned relative to nearest positioned ancestor.
fixed	Like absolute but relative to the browser window.
sticky	Normal flow until scroll triggers stickiness.

4.7 Other Important CSS Properties

- **Colors:** name, #RRGGBB, rgb(), hsl() — color is inherited
- **Fonts:** font-family (with fallback), font-size (use em), font-weight, font-style
- **Text:** text-align, text-decoration, text-transform, letter-spacing, word-spacing
- **Background:** background-color, background-image, background-repeat, background-position
- **Lists:** list-style-type, list-style-position, list-style-image
- **Overflow:** visible (default), hidden, scroll, auto
- **Measurement units:** px, em (recommended, 1em=16px), %, cm, mm, pt

5. JavaScript

5.1 Introduction

JavaScript is a **high-level, dynamic, untyped, interpreted** programming language. It is the only language natively supported by all web browsers for client-side development. JavaScript $\neq$ Java. JavaScript = ECMAScript.

How to Load JavaScript

Key Concept: Exam Q3: An external JS file is loaded via: `<script src="myScript.js"></script>`

- **External file:** `<script src="myScript.js"></script>` (recommended)
- **Inline in page:** `<script>...code...</script>`
- **Event handler attribute:** `onclick="myFunction()"`

5.2 Variables and Types

- Declare with **let** (modern) or **var** (older)
- JavaScript is **case-sensitive** and **untyped** (variable types can change at runtime)
- **Scope:** Global (accessible everywhere) or Local (within a function)
- Variable names: cannot start with number, cannot use reserved words, case-sensitive

Data Types

- **Primitive:** Numbers (no int/float distinction), Strings, Booleans, null, undefined
- **Object:** Objects (key-value collections), Arrays (ordered), Functions
- **null** = expected absence of value; **undefined** = unexpected/error absence
- **Falsy values:** false, 0, "", null, undefined, NaN

5.3 Operators

- **Arithmetic:** +, -, *, /, %, ++, --
- **Comparison:** ==, !=, >, <, >=, <= (== does type coercion!)
- **Strict equality:** === (no type coercion — values must be same type AND value)
- **Logical:** && (AND), || (OR), ! (NOT)
- **Ternary:** condition ? valueIfTrue : valueIfFalse
- **typeof:** Returns the type of a variable as a string
- **String +:** Concatenation ("a" + 10 $\rightarrow$ "a10")

5.4 Control Structures

- **if/else:** `if (condition) { } else { }`
- **for loop:** `for (let i=0; i<n; i++) { }`
- **while:** `while (condition) { }`
- **do/while:** `do { } while (condition);` — executes at least once

- **for/in**: Iterates over object properties
- **switch**: `switch(n) { case 1: ...; break; default: ...; }`

5.5 Functions

```
// Function declaration function sum(a, b) { return a + b; } // Function expression var
sum = function(a, b) { return a + b; } // Arrow function (ES6) var sum = (a, b) => {
return a + b; }
```

- Functions can be assigned to variables and passed as arguments
- Functions can be defined at runtime

5.6 The DOM (Document Object Model)

The DOM is the **in-memory browser representation** of a web page as a **tree of objects**. JavaScript can manipulate the DOM to create dynamic content.

Finding Elements

```
document.getElementById("myId") document.getElementsByTagName("p")
document.getElementsByClassName("myClass") document.querySelectorAll("p.myClass")
```

Changing Elements

```
element.innerHTML = "new content" element.style.color = "red"
element.setAttribute("hidden", false)
```

Adding/Removing Elements

```
document.createElement("p") document.removeChild(element) parent.appendChild(newChild)
```

■ **Exam Tip**: DOM manipulation must happen AFTER the elements exist. Use **window.onload** or place `<script>` at the end of `<body>`.

5.7 Event Handlers

- **onclick** — buttons, submit
- **onchange** — text fields, checkboxes, radio buttons, select
- **window.onload** — when page finishes loading

```
window.onload = function() { document.getElementById("myp").innerHTML = "Hello"; }
```

6. jQuery, Bootstrap & AJAX

6.1 jQuery

jQuery is an open-source JavaScript library that simplifies DOM manipulation, event handling, and AJAX.

Syntax: `$(selector).action()`

Selecting Elements

```
$("td"); // all td elements $("#contacts"); // element with id "contacts"
$(".controls"); // elements with class "controls" $("body h1"); // h1 elements inside body
```

Actions

```
$("p").hide() // hide all p elements $("h1").text() // get text content
$("h1").text("hello") // set text content $("#myid").html("new") // set HTML content
$("p").css("color", "red") // change CSS property
```

DOM Manipulation

- **before()/after()**: Insert adjacent to selected element
- **append()/prepend()**: Add as child of selected element
- **addClass()/removeClass()**: Modify CSS classes

Events

```
$("#button").click(function() { ... }); $(document).ready(function() { ... }); // "this"
refers to the element that triggered the event $("#button").click(function() {
$(this).text("Clicked!"); });
```

■ **Exam Tip: Arrow functions** do NOT have their own 'this'. They inherit it from the enclosing scope. Use **event.target** instead in arrow functions.

6.2 Bootstrap

Bootstrap is a free CSS framework for responsive, mobile-first web development. Bootstrap 5 removed jQuery dependency.

Grid System (12 columns)

- Based on **containers**, **rows**, and **columns**
- `.container` (fixed width) vs `.container-fluid` (full width)
- Column classes: **.col-*-N** where * = breakpoint, N = columns out of 12
- Breakpoints: xs(<576px), sm(≥576), md(≥768), lg(≥992), xl(≥1200)
- Below a breakpoint, columns stack vertically

6.3 AJAX

Key Concept: AJAX = **Asynchronous JavaScript And XML**. Allows updating pages without reloading. Modern implementations use **JSON** instead of XML.

- Update page without reload; request/send data in background

- **XMLHttpRequest** object sends requests to server
- Server processes and sends response; JS reads response and updates page

```
$.ajax({ type: "GET", url: url, dataType: "json", success: function(response) { //  
handle response } });
```

JSON (JavaScript Object Notation)

- Lightweight data exchange format; easy for humans and JS interpreters
- Objects: { "key": "value" } — unordered name/value pairs
- Arrays: [value1, value2] — ordered collections
- Values: strings, numbers, booleans, null, objects, arrays (can be nested)

Single Page Application (SPA)

- Dynamically rewrites current page instead of loading new pages
- Single HTML file, heavy use of JavaScript, improved UX
- Examples: Gmail, Google Maps, Facebook

7. HTTP Protocol & Flask Web Server

7.1 HTTP Fundamentals

HTTP is an **application-level, stateless, request-response, pull protocol**. The client pulls information from the server.

HTTP Methods

Method	Purpose	Safe	Idempotent	Has Body
GET	Read/retrieve data	Yes	Yes	No
POST	Create a new resource	No	No	Yes
PUT	Update/replace a resource	No	Yes	Yes
DELETE	Delete a resource	No	Yes	No
HEAD	Get headers only (no body)	Yes	Yes	No
PATCH	Partial modification	No	No	Yes
OPTIONS	Query supported methods	Yes	Yes	No

Key Concept: Exam Q4: The primary purpose of POST is **to create or add a new item**. POST is NOT idempotent (two identical requests create duplicates). GET is read-only and idempotent.

HTTP Status Codes

- **200 OK:** Request successful
- **201 Created:** Resource created (POST success)
- **302 Redirect:** Go somewhere else
- **403 Forbidden:** Access denied
- **404 Not Found:** Resource not found
- **500 Server Error:** Internal server error

7.2 Flask

Flask is a Python **microframework** for web development. Lightweight, flexible, ready to go.

```
from flask import Flask, render_template, request
app = Flask(__name__)
@app.route("/")
def hello():
    return "<h1>Hello World!</h1>"
@app.route("/user/<username>")
def show_user(username):
    return render_template("user.html", name=username)
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        user = request.form.get("username")
    ...
```

- **@app.route()** maps URLs to functions
- **URL variables:** <name> or <int:age> for typed parameters
- **Static files:** Served automatically from /static/ folder
- **Templates:** Use render_template() with Jinja2 from /templates/ folder
- **Request object:** request.method, request.form, request.args
- Default address: **127.0.0.1:5000**

8. Cookies, Sessions & Authentication

8.1 Sessions & Cookies

HTTP is **stateless** — each request is independent. **Cookies** enable statefulness by storing key-value pairs in the browser that are sent with subsequent requests.

Cookie Purposes

- **Session management:** Login status, shopping cart, game scores
- **Personalization:** User preferences, language, theme
- **Tracking:** Recording and analyzing user behavior

Cookie Types

- **Session cookies:** Deleted when browser closes (no Expires/Max-Age)
- **Persistent cookies:** Have explicit Expires or Max-Age attribute

Cookie Header Attributes

```
Set-Cookie: name=value; Expires=date; Path=/; Domain=example.com; Secure; HttpOnly; SameSite
```

- **Expires/Max-Age:** When the cookie expires
- **Path:** Restricts cookie to specific URL paths
- **Domain:** Specifies which domain(s) receive the cookie
- **Secure:** Only sent over HTTPS
- **HttpOnly:** Cannot be accessed via JavaScript (prevents XSS)
- **SameSite:** Protects against CSRF attacks

Cookies in Flask

```
# Setting a cookie resp = make_response(render_template("page.html"))
resp.set_cookie("userID", "alice", max_age=3600) return resp # Reading a cookie user =
request.cookies.get("userID")
```

8.2 Authentication

Basic Authentication

- Username:password encoded with **Base64**
- **Insecure!** Base64 is trivially reversible (same as no encryption)

Digest Authentication

- Uses a **hash function** (MD5, SHA-256) instead of sending password
- Hash properties: quick to compute, irreversible, tamper-resistant, collision-resistant
- **Problem:** Replay attacks (same password → same digest)
- **Solution:** Use a **nonce** (number used only once) combined with password

9. Web Security

9.1 HTTPS (HTTP + SSL/TLS)

Key Concept: Exam Q5: SSL/TLS purpose is **to provide secure communication**. HTTPS adds a security layer (SSL/TLS) between HTTP and TCP.

- **SSL** = Secure Sockets Layer; **TLS** = Transport Layer Security
- Uses **asymmetric cryptography** (public + private key pairs)
- **Certificates** issued by Certificate Authorities (CAs) for verification
- **SSL handshake:** Browser gets certificate → extracts public key → generates secret key → encrypts with public key → sends to server

9.2 Cookie Security Threats

- **Network threats:** Eavesdropping on unencrypted channels → use HTTPS/Secure flag
- **End-system threats:** Cookies stolen/copied from local storage → cache sniffing
- **Cookie harvesting:** Attacker impersonates website to collect cookies
- **Cross-Site Scripting (XSS):** Injecting malicious scripts via cookies

Mitigations

- Use **Secure** flag (HTTPS only)
- Use **HttpOnly** flag (block JavaScript access)
- Use **SameSite** attribute (prevent CSRF)
- Encrypt sensitive cookie data
- Comply with regulations: GDPR, ePrivacy Directive

9.3 Password Security

- Never store plaintext passwords — store **hashes**
- Use **salts** (random values unique per user) to prevent identical hashes

```
import hashlib, random
salt = str(random.random())
digest = hashlib.sha256((salt + password).encode()).hexdigest()
```


10. Databases & SQL

10.1 Database Concepts

- **Entity:** Something to record facts about (student, exam)
- **Property:** Attribute of an entity (name, email)
- **Relationship:** Link between entities (student takes exam)
- **Table:** Represents an entity; rows are instances, columns are properties
- **Key:** Minimal set of columns that uniquely identifies a row
- **Primary key:** Chosen key for the table (auto-indexed)
- **Foreign key:** References a primary key in another table

10.2 SQL Sublanguages

- **DDL** (Data Definition): CREATE TABLE, ALTER TABLE, DROP TABLE
- **DQL** (Data Query): SELECT ... FROM ... WHERE ...
- **DML** (Data Manipulation): INSERT INTO, UPDATE, DELETE
- **DCL** (Data Control): GRANT, REVOKE

SQL is a **declarative** language — you describe WHAT you want, not HOW to get it.

Key SQL Commands

```
-- Create table CREATE TABLE student ( name CHAR(256) NOT NULL, student_ID CHAR(8)
PRIMARY KEY, email CHAR(256) CHECK (email LIKE '%@%') ); -- Insert data INSERT INTO
student VALUES('Alan', '00002', 'alan@yahoo.com'); -- Query data SELECT name FROM student
WHERE student_ID = '00001'; -- JOIN tables SELECT s.name FROM student s JOIN exam e WHERE
s.student_ID = e.student_ID AND e.grade = 30;
```

10.3 SQLite & Prepared Statements

- SQLite is serverless — uses a local file, no network
- **Prepared statements** prevent SQL injection:

```
cursor.execute("SELECT * FROM user WHERE username=? AND password=?", (username, digest))
```

11. Jinja2 Templating

Jinja2 is a fast, secure Python templating engine used with Flask.

Delimiters

- `{{ ... }}` — Expressions (output to template)
- `{% ... %}` — Statements (if, for, block, extends)
- `{# ... #}` — Comments (not in output)

Template Inheritance

```
<!-- base.html --> <html> <body> {% block content %}{% endblock %}
</body> </html> <!-- child.html --> {% extends "base.html" %} {% block
content %} <h1>Hello {{ name }}!</h1> {% endblock %}
```

MOCK EXAM — Questions & Answers

Format: 10 Multiple-Choice Questions (select one) + 6 Open-Ended Questions. Based on the official 2024-12-17 sample exam plus additional questions covering all topics.

Section A: Multiple-Choice Questions

For each question, select ONE correct answer from the five provided.

Question 1) What does Web 3.0 aim to achieve?

- A. Development of static web pages
- B. Making Internet data machine-readable
- C. Enhancing the HTML language
- D. Increasing web browser competition
- E. Reinventing file transfer protocols

✓ **Correct Answer: B — Web 3.0 (the Semantic Web) aims to make data machine-readable, enabling machines to analyze and understand web content.**

Question 2) What is the purpose of CSS?

- A. To add interactivity to web pages
- B. To define the structure of documents
- C. To separate content from presentation
- D. To execute tasks concurrently
- E. To improve user experience by loading data asynchronously

✓ **Correct Answer: C — CSS separates content (HTML) from presentation (visual styling), allowing different styles for different media and easier maintenance.**

Question 3) How is an external JavaScript file loaded into an HTML document?

- A. `<script>myScript.js</script>`
- B. `<script src="myScript.js"></script>`
- C. `<link rel="script" href="myScript.js">`
- D. `<javascript src="myScript.js"></javascript>`
- E. `<import src="myScript.js"></import>`

✓ **Correct Answer: B — The `<script>` tag with the `src` attribute is the standard way to load external JavaScript files.**

Question 4) What is the primary purpose of the POST method in HTTP?

- A. To retrieve data
- B. To delete data
- C. To create or add a new item
- D. To update a resource
- E. To read the server response

✓ **Correct Answer: C — POST is used to create or add new resources. It is NOT idempotent (two identical POST requests create duplicates).**

Question 5) What is the purpose of SSL and TLS?

- A. To block cookies
- B. To provide secure communication
- C. To speed up data transmission
- D. To compress web pages
- E. To enable user tracking

✓ **Correct Answer: B** — SSL/TLS are cryptographic protocols that provide secure, encrypted communication between client and server (forming HTTPS).

Question 6) What is the correct CSS specificity value for the selector p.bright em.dark?

- A. 0,0,0,2
- B. 0,0,1,2
- C. 0,0,2,2
- D. 0,1,0,2
- E. 0,0,2,0

✓ **Correct Answer: C** — There are 2 element selectors (p, em) = 2 points, and 2 class selectors (.bright, .dark) = 20 points. Total: 0,0,2,2.

Question 7) In the TCP/IP model, which layer is responsible for routing packets to their destination?

- A. Physical layer
- B. Data Link layer
- C. Network layer
- D. Transport layer
- E. Application layer

✓ **Correct Answer: C** — The Network layer handles routing, addressing, and congestion avoidance, ensuring packets reach their correct destination.

Question 8) What is the result of the JavaScript expression: typeof NaN?

- A. "NaN"
- B. "undefined"
- C. "number"
- D. "string"
- E. "object"

✓ **Correct Answer: C** — NaN (Not a Number) is still of type "number" in JavaScript. It represents a number that a computer cannot calculate.

Question 9) In Flask, what does the @app.route() decorator do?

- A. It defines a CSS stylesheet for the route
- B. It maps a URL pattern to a Python function
- C. It creates a database table for the route
- D. It encrypts the data sent through the route
- E. It validates HTML content returned by the function

✓ **Correct Answer: B** — The @app.route() decorator tells Flask which URL should trigger the decorated function. The function returns the response (usually HTML).

Question 10) Which cookie attribute prevents JavaScript from accessing the cookie via document.cookie?

- A. Secure

- B. SameSite
- C. HttpOnly
- D. Path
- E. Domain

✓ **Correct Answer: C** — The HttpOnly attribute restricts the cookie from being accessed via JavaScript (e.g., `document.cookie`), which helps mitigate XSS attacks.

Section B: Open-Ended Questions

Answer each question thoroughly, demonstrating understanding of concepts.

Question 1) In what ways does separating structure from presentation facilitate better web design and user experience?

Model Answer: Separating structure (HTML) from presentation (CSS) provides several key advantages:

- 1. Division of labor:** Authors/programmers can focus on content while designers handle visual presentation independently. Designers can enforce consistent organizational styles across all documents.
- 2. Flexibility and maintainability:** When markup contains only content and structure, the appearance can be quickly changed. Different presentation settings can be defined for different media devices (screen, print, mobile). The style of an entire website can be changed without rewriting content.
- 3. Accessibility:** Documents can be presented by Braille printers for those with limited vision, or by text readers for those with limited hearing. Screen readers can interpret the semantic structure without being confused by presentation markup.
- 4. Performance:** External CSS files can be cached by browsers, reducing bandwidth and load times. Multiple pages sharing the same stylesheet only download it once.

Question 2) What is asynchronous programming? Discuss the role of AJAX in creating dynamic web applications.

Model Answer: **Asynchronous programming** is a paradigm that allows a program to perform tasks concurrently without blocking the execution flow. Key concepts include:

Concurrency: Tasks can start, run, and complete in overlapping time periods.

Non-blocking: Execution continues even while an operation is in progress, maintaining application responsiveness.

AJAX (Asynchronous JavaScript And XML) is a set of techniques for creating asynchronous web applications. Its role includes:

- 1. Background data exchange:** AJAX allows retrieving or sending data to a web server asynchronously without interfering with the display of the existing page.
- 2. No page reload:** Web pages can update content, request data, and send data without full page reloads.
- 3. How it works:** An event triggers JavaScript to create an XMLHttpRequest object, which sends a request to the server. The server processes it and sends a response. JavaScript reads the response and performs actions (e.g., updates the DOM).
- 4. Modern format:** Originally used XML, but modern implementations use JSON for its simplicity and native JavaScript compatibility.
- 5. Enables SPAs:** Single Page Applications like Gmail rely on AJAX for dynamic content without navigation.

Question 3) What is the purpose of cookies? What are some common security threats associated with cookies? How can they be mitigated?

Model Answer: Purpose of cookies: Cookies are small key-value pairs stored by the browser on behalf of the server. They are sent via HTTP headers (invisible to users). Their main purposes are:

- **Session management:** Login status, shopping carts, game scores
- **Personalization:** User preferences, language settings, themes
- **Tracking:** Recording and analyzing user behavior

Security threats:

1. **Network threats (Eavesdropping):** Cookies sent over unencrypted HTTP can be intercepted.
2. **End-system threats:** Cookies stored in cleartext on the local drive can be stolen, copied, or modified (cache sniffing).
3. **Cookie harvesting:** Attackers impersonate websites to collect cookies from users, then use them on legitimate sites.
4. **Cross-Site Scripting (XSS):** Malicious scripts injected via URLs can steal cookie data.
5. **Third-party tracking:** Embedded ads set cookies that track users across websites.

Mitigations:

- Use **Secure** flag (cookies only sent over HTTPS)
- Use **HttpOnly** flag (blocks JavaScript access via document.cookie)
- Use **SameSite** attribute (prevents CSRF attacks)
- Encrypt sensitive cookie data
- Comply with GDPR, ePrivacy Directive regulations
- Allow users to opt out of non-essential cookies

Additional Open-Ended Questions

Question 4) Explain the CSS Box Model. How do the content, padding, border, and margin areas relate to each other? Why are box-model properties NOT inherited?

Model Answer: The CSS **Box Model** defines that every HTML element generates a rectangular box composed of four concentric areas (from innermost to outermost):

- 1. Content:** The actual content of the element (text, images). Its dimensions are set with the width and height properties.
- 2. Padding:** Transparent space between the content and the border. Can be set uniformly (padding: 10px) or per side (padding-top, padding-bottom, padding-left, padding-right). Background color extends into the padding area.
- 3. Border:** Lines surrounding the padding, defined by border-width, border-style, and border-color. Transparent parts of dashed/dotted borders show the background color.
- 4. Margin:** Transparent space outside the border, creating distance between elements. Margins can collapse (adjacent vertical margins merge to the largest value).

Why NOT inherited: If box-model properties were inherited, every child element would receive the same margins, borders, and padding as its parent, creating undesirable cascading layout effects. For example, if a <div> had margin: 20px and this was inherited, every <p>, , and nested element would also have 20px margins, making layouts impossible to control.

Question 5) Describe the Document Object Model (DOM). How does JavaScript interact with it to create dynamic web pages? Why must DOM manipulation occur after the page has loaded?

Model Answer: The **Document Object Model (DOM)** is the in-memory browser representation of a web page as a **tree of objects**. The HTML elements are represented as nodes in this tree, accessible through the window.document property.

Four types of DOM objects:

- **Document:** Represents the entire document (window.document)
- **Element:** Represents HTML tags
- **Attr:** Represents attributes of elements
- **Text:** Represents text content within elements

JavaScript interacts via the DOM API:

- **Selecting nodes:** getElementById(), getElementsByClassName(), querySelectorAll()
- **Changing content:** element.innerHTML, element.style, element.setAttribute()
- **Adding/removing nodes:** createElement(), appendChild(), removeChild()
- **Event handling:** onclick, onchange, addEventListener()

Why after loading: When a browser parses HTML, it builds the DOM top-to-bottom. If a script in <head> tries to access an element in <body>, that element doesn't exist yet, causing a null reference error. Solutions: (1) Place <script> at end of <body>, or (2) Use window.onload = function() { ... } to wait for complete page load.

Question 6) Compare and contrast the HTTP methods GET and POST. When should each be used? Explain the concepts of 'safe' and 'idempotent' in this context.

Model Answer:

GET method:

- Purpose: Retrieve/read data from the server
- Data sent via URL query string (visible in address bar)
- **Safe:** Yes — has no effect on the server (read-only)
- **Idempotent:** Yes — multiple identical requests return the same result
- **Cacheable:** Yes — responses can be cached
- No request body
- Use cases: Retrieving web pages, search queries, API data fetching

POST method:

- Purpose: Create or add new resources to the server
- Data sent in the request body (not visible in URL)
- **Safe:** No — modifies server state
- **Idempotent:** No — two identical POST requests create duplicate resources
- Returns status 201 (CREATED) on success
- Has request body
- Use cases: Submitting forms, creating accounts, uploading files

Safe means the method has no side effects on the server (read-only). **Idempotent** means making the same request multiple times produces the same result. GET is both safe and idempotent. POST is neither. PUT is idempotent but not safe (it modifies data, but repeated PUTs produce the same state).

End of Study Guide

Good luck on your exam! Remember: the written test is based strictly on the course slides. This guide covers all 15 lectures (469 slides) comprehensively.

Lab of Web Technologies — Ca' Foscari University of Venice — AY 2025-2026